Taller: Uso de Logs en una API REST con Spring Boot, Spring Data JPA y PostgreSQL

Objetivo:

Aprender a implementar logs efectivos en una API REST usando Spring Boot, Spring Data JPA y PostgreSQL, aplicando SLF4J, Logback y Logs en Base de Datos para depuración y monitoreo.

Duración:

3 horas

Requisitos Previos:

JDK 17 o superior instalado
PostgreSQL 14 o superior instalado y configurado
IDE como IntelliJ IDEA, Eclipse o VS Code con Extensión de Java
Postman o Swagger para probar la API
Conocimientos básicos de Spring Boot, Spring Data JPA y PostgreSQL

Parte 1: Configuración del Proyecto y PostgreSQL

Ejercicio 1: Creación del Proyecto

Objetivo: Configurar Spring Boot, Spring Data JPA y PostgreSQL con Logs.

Paso a Paso:

1. **Ve a Spring Initializr y genera un nuevo proyecto con:**
    - **Maven, Java, Spring Boot 3.x.x**
    - **Dependencias:**
      Spring Web
      Spring Boot DevTools
      Spring Data JPA
      PostgreSQL Driver
      Lombok
      Spring Boot Logging
2. **Descarga y descomprime el proyecto.**
3. **Abre el proyecto en tu IDE (IntelliJ o Eclipse).**

Ejercicio 2: Configuración de PostgreSQL

1. Crea una base de datos llamada log_db.
2. En src/main/resources/application.properties, configura la conexión:

```
spring.datasource.url=jdbc:postgresql://localhost:5432/log_db
spring.datasource.username=tu_usuario
spring.datasource.password=tu_contraseña
spring.datasource.driver-class-name=org.postgresql.Driver

spring.jpa.database-platform=org.hibernate.dialect.PostgreSQLDialect
spring.jpa.hibernate.ddl-auto=update
spring.jpa.show-sql=true

# Configuración de Logs en consola y archivo
logging.level.root=INFO
logging.level.org.springframework=DEBUG
logging.file.name=logs/api.log
```

Resultado esperado: PostgreSQL configurado y logs habilitados en consola y archivo.

Parte 2: Implementación de Logs en la API

Ejercicio 3: Creación de la Entidad y el Repositorio

1. **Crea un paquete models y agrega Producto.java:**

```
package com.miempresa.api.models;

import jakarta.persistence.*;
import lombok.*;

@Entity
```

```
@Table(name = "productos")
@Getter
@Setter
@NoArgsConstructor
@AllArgsConstructor
public class Producto {
  @Id
  @GeneratedValue(strategy = GenerationType.IDENTITY)
  private Long id;

  @Column(nullable = false)
  private String nombre;

  private double precio;
}
```

2. Crea un paquete repositories y agrega ProductoRepository.java:

```
package com.miempresa.api.repositories;

import com.miempresa.api.models.Producto;
import org.springframework.data.jpa.repository.JpaRepository;

public interface ProductoRepository extends JpaRepository<Producto, Long> {
}
```

Resultado esperado: Creación de la entidad y repositorio con PostgreSQL.

Ejercicio 4: Creación del Servicio con Logs

1. Crea un paquete services y agrega ProductoService.java:

```
package com.miempresa.api.services;


import com.miempresa.api.models.Producto;
import com.miempresa.api.repositories.ProductoRepository;
import lombok.extern.slf4j.Slf4j;
import org.springframework.stereotype.Service;


import java.util.List;


@Service
@Slf4j
public class ProductoService {
  private final ProductoRepository productoRepository;


  public ProductoService(ProductoRepository productoRepository) {
    this.productoRepository = productoRepository;
  }


  public List<Producto> obtenerProductos() {
    log.info("Obteniendo lista de productos");
    return productoRepository.findAll();
  }


  public Producto guardarProducto(Producto producto) {
    log.info("Guardando producto: {}", producto.getNombre());
```

```
    return productoRepository.save(producto);
  }

  public void eliminarProducto(Long id) {
    log.warn("Eliminando producto con ID: {}", id);
    productoRepository.deleteById(id);
  }
}
```

Resultado esperado: Implementación de logs en información, advertencias y errores en el servicio.

Ejercicio 5: Creación del Controlador con Logs

1. **Crea un paquete controllers y agrega ProductoController.java:**

```
package com.miempresa.api.controllers;

import com.miempresa.api.models.Producto;
import com.miempresa.api.services.ProductoService;
import lombok.extern.slf4j.Slf4j;
import org.springframework.web.bind.annotation.*;

import java.util.List;

@RestController
@RequestMapping("/productos")
@Slf4j
public class ProductoController {
  private final ProductoService productoService;

  public ProductoController(ProductoService productoService) {
    this.productoService = productoService;
  }

  @GetMapping
  public List<Producto> obtenerProductos() {
    log.info("Solicitud GET para obtener productos");
    return productoService.obtenerProductos();
  }
```

```
  @PostMapping
  public Producto guardarProducto(@RequestBody Producto producto) {
    log.info("Solicitud POST para guardar producto: {}",
producto.getNombre());
    return productoService.guardarProducto(producto);
  }

  @DeleteMapping("/{id}")
  public void eliminarProducto(@PathVariable Long id) {
    log.warn("Solicitud DELETE para eliminar producto con ID: {}", id);
    productoService.eliminarProducto(id);
  }
}
```

Resultado esperado: API con logs en controladores y servicios.

Parte 3: Configuración Avanzada de Logs

Ejercicio 6: Configuración de Logs con Logback

1. **Crea el archivo src/main/resources/logback-spring.xml:**

```
<configuration>
  <appender name="FILE" class="ch.qos.logback.core.FileAppender">
    <file>logs/api.log</file>
    <encoder>
      <pattern>%d{yyyy-MM-dd HH:mm:ss} - [%level] - %msg%n</pattern>
    </encoder>
  </appender>

  <appender name="CONSOLE"
class="ch.qos.logback.core.ConsoleAppender">
    <encoder>
      <pattern>%d{yyyy-MM-dd HH:mm:ss} - [%level] - %msg%n</pattern>
    </encoder>
  </appender>

  <logger name="com.miempresa.api" level="DEBUG" additivity="false">
    <appender-ref ref="FILE"/>
    <appender-ref ref="CONSOLE"/>
  </logger>
  <root level="INFO">
    <appender-ref ref="CONSOLE"/>
  </root>
</configuration>
```

Resultado esperado: Logs guardados en logs/api.log y visibles en consola.

Parte 4: Pruebas y Monitoreo de Logs

Ejercicio 7: Prueba de Logs en la API

1. **Ejecuta la API con:**

mvn spring-boot:run

2. **En Postman o Swagger, prueba los endpoints:**
   - **GET /productos**
   - **POST /productos**
   - **DELETE /productos/{id}**
3. **Verifica los logs en consola y en logs/api.log.**

Resultado esperado: Logs generados con cada solicitud.

Parte 5: Reto Final

Desafío:

1. Agregar logs en el manejo de excepciones.
2. Configurar un log en base de datos con PostgreSQL.
3. Configurar logs para enviar alertas en caso de errores críticos.

Conclusión

Implementamos logs efectivos en Spring Boot.
Configuramos Logback y logs en archivo.
Probamos la API con logs en cada solicitud.

Siguientes pasos: Integrar monitorización con ELK Stack (Elasticsearch, Logstash, Kibana).